Survey Forms (Google Forms Clone)

Build Guide: Project Creation, File Map, Integration and Requirements

Written by Kasinadh Sarma

13 October 2026

Contents

1 Overview **2**

2 Requirements **2**

2.1 Software . . . 2

2.2 Packages . . . 2

2.3 Why these libraries . . . 3

2.4 Accounts and configuration . . . 4

3 How the Layers Connect **5**

4 Step-by-Step Build **5**

5 What Was Verified **11**

6 Known Limits **11**

7 Source Code **12**

7.1 Configuration . . . 12

7.2 Shared API schemas . . . 12

7.3 Server foundations . . . 13

7.4 Repository layer . . . 13

7.5 API routes . . . 14

7.6 Client library . . . 14

7.7 Components . . . 14

7.8 Pages . . . 15

1 Overview

This guide explains, in order, how to create the project, which files are created or changed at each step, and how the layers connect. The application lets a user build a survey, share a public link, collect validated responses, and view the submissions.

Stack. Next.js (App Router), Tailwind CSS v4, react-hook-form, Zod, `@better-fetch/fetch`, Hono (inside a Next.js route handler), Prisma 7 with PostgreSQL as the primary database, and Cloud Firestore (through the Firebase Admin SDK) as a live mirror of that data and as the store for uploaded files.

Project layout. The project uses the root `app/` layout (no `src/` folder). Section 5 says exactly what was checked and what was not.

Core idea. The API Zod schemas, kept one file per concern in `lib/schemas/`, are imported by both the browser and the API. The same rules therefore run in the form, in the Hono validator, and before anything reaches a database.

Decisions in this version.

- **Hono, not Elysia.** Hono runs inside a Next.js route handler, giving one project and one deployment.
- **Postgres is the source of truth; Firestore mirrors it.** Every write goes to Postgres through Prisma first and is then copied to Firestore under the same ids. If Postgres is unreachable, reads fall back to Firestore. Section 3 explains the design and its limits.
- **Files live in Firestore, not in a storage bucket.** A new Firebase bucket may need a paid plan, so uploaded files are stored as documents in a Firestore collection. This caps a file at 700 KB (Firestore documents are limited to 1 MiB). Moving to a real bucket later changes only `lib/server/repo/files.ts` and the two file routes.

2 Requirements

2.1 Software

Requirement	Version	Purpose
Node.js	22 or newer (24.18 used here)	Runtime
npm	10 or newer (11.16 used here)	Package manager
PostgreSQL	Local, Docker, Neon or Supabase	Primary database used by Prisma
Firebase project	Cloud Firestore enabled, Admin SDK service account	Mirror database and file store
Git	Any recent version	Version control

2.2 Packages

These are the versions installed in this project on 13 October 2026, read from `node_modules`.

Package	Version	Role
`next`	16.4.0	Framework (Turbopack, Cache Components on by default)
`react`, `react-dom`	19.3.0	UI
`tailwindcss`, `@tailwindcss/postcss`	4.3.3	Styling (v4, CSS-first)
`postcss`	8.5.30	Used by the Tailwind plugin
`zod`	4.6.7	Shared schemas and types
`react-hook-form`	7.90.0	Form state
`@hookform/resolvers`	5.9.1	Connects Zod to react-hook-form
`@better-fetch/fetch`	1.3.2	Typed HTTP client
`hono`	4.13.13	API framework
`@hono/zod-validator`	0.9.1	Request validation middleware
`@prisma/client`, `prisma`	7.10.0	Database client, migrations and generation
`@prisma/adapter-pg`, `pg`	7.10.0, 8.23.1	Postgres driver adapter (required by Prisma 7)
`firebase-admin`	14.5.0	Firestore access from the server
`dotenv`	18.0.7	Loads `.env` for `prisma.config.ts`
`typescript`, `eslint`, `prettier`	5.9.3, 9.39.5, 3.9.10	Types, lint, formatting

2.3 Why these libraries

Each package is here for one job, and the list is kept short on purpose.

Package	Why it is used
`next`, `react`, `react-dom`	One framework for pages, routing and server rendering, and the host for the API route handler.
`tailwindcss`, `@tailwindcss/postcss`, `postcss`	Utility-first styling without a separate stylesheet system. v4 is CSS-first, and the PostCSS plugin keeps styling independent of the bundler.
`zod`	One schema per concern, shared by the browser and the API, so the form, the validator and the database layer apply the same rules and the TypeScript types are inferred from them.
`react-hook-form`	Fast form state for the survey builder and response forms, with few re-renders.
`@hookform/resolvers`	Lets react-hook-form use the Zod schemas directly instead of duplicating rules.
`@better-fetch/fetch`	A TypeScript-first wrapper over native `fetch`: typed responses and one error shape, without the weight of Axios.
`hono`	A small, fast web framework that runs inside a Next.js route handler, so the API ships in the same project and deployment.
`@hono/zod-validator`	Validates request bodies, params and queries in Hono with the shared Zod schemas.
`@prisma/client`, `prisma`	A type-safe database client generated from the schema, plus migrations.
`@prisma/adapter-pg`, `pg`	Prisma 7 requires a driver adapter; this is the node-postgres one for PostgreSQL.
`firebase-admin`	Server-side access to Firestore, used as the live mirror and file store.
`dotenv`	Loads `.env` for `prisma.config.ts`, which Next.js does not do.
`typescript`, `eslint`, `prettier`	Static types, lint rules and consistent formatting.

Prisma versions must match. The npm `latest` tag of `prisma` pointed to an 8.0 release candidate while `@prisma/client` was 7.10.0. The install commands in Step 2 pin the three Prisma packages to 7.10.0. Change all three together when you upgrade.

2.4 Accounts and configuration

- A PostgreSQL connection string (`DATABASE_URL`).
- A Firebase project with a Cloud Firestore database (the `(default)` database; the free Spark plan is enough).
- A service account key for the Firebase Admin SDK: project id, client email and private key (Firebase console, Project settings, Service accounts, Generate new private key).
- A random token for the sync endpoints (`SYNC_TOKEN`).

3 How the Layers Connect

Layer	What it does
`components/`	Forms, tables and inputs. Validate with the shared Zod schemas before sending.
`lib/fetch.ts`	Typed client; every endpoint has input, output, params and query schemas.
`lib/server/routes/`	Hono routes. Validate again on the server; return one error shape.
`lib/server/repo/forms.ts`, `submissions.ts`	The only entry points the routes use. They combine the two databases.
`lib/server/repo/postgres/`	Prisma code: the source of truth.
`lib/server/repo/firestore/`	Firestore code: the mirror and the read fallback.
`lib/server/repo/files.ts`	Uploaded files, stored in Firestore.
`lib/server/repo/sync.ts`	Compares and repairs the two databases.

Writes. `createForm`, `deleteForm` and `createSubmission` run in Postgres first. Only after Postgres has committed does the same row go to Firestore, under the same id. A failure of the Firestore write is logged and does not fail the request, because the response is already correct; the sync endpoint repairs the difference.

Reads. Reads come from Postgres. If Postgres throws (for example it is unreachable), the same read runs against Firestore. A "not found" from Postgres is a real answer and is not retried.

Firestore layout. A form is one document in `forms/{id}` with its questions embedded and a `responseCount`. Each submission is a document in the subcollection `forms/{id}/submissions/{id}`. Uploaded files are documents in `files/{id}`.

Counter. When a submission is mirrored, a transaction writes the document and increments `responseCount` only if the document did not exist, so mirroring the same submission twice counts once.

Limits of this design.

- Sync is one-way, from Postgres to Firestore. Edits made directly in Firestore are overwritten by the next sync.
- When Postgres is down, reads work but new forms and submissions fail.
- Syncing is manual (an endpoint you call); there is no scheduler.

4 Step-by-Step Build

Step 1: Create the Next.js project with Tailwind v4

```
npx create-next-app@latest survey-forms --ts --tailwind --eslint --app --no-src-dir --use-npm --
    import-alias "@/*"
cd survey-forms
```

Files created by the generator: `package.json`, `tsconfig.json`, `next.config.ts`, `eslint.config.mjs`, `AGENTS.md`, `README.md`, and the folders `app/` and `public/`.

No `src/` folder. Every folder in this guide (`app/`, `components/`, `lib/` (with `lib/server/` and `lib/schemas/` inside), `prisma/`, `generated/`) sits at the project root. The `@/` alias points to the root through `"paths": { "@/*": ["./*"] }` in `tsconfig.json`, which the starter writes for you.

Tailwind v4. `app/globals.css` begins with `@import "tailwindcss";`. There are no `@tailwind` directives and no `tailwind.config.js`. Create `postcss.config.mjs` (listing in Section 7) so the styling does not depend on the starter's Turbopack rule.

Cache Components. The starter sets `cacheComponents: true`. Two rules follow, and the code already applies both. A route handler must not export `runtime`. Any page that awaits `params`, and any client component that creates random ids (react-hook-form's field array does), must sit inside a `Suspense` boundary, or `next build` fails.

Step 2: Install dependencies

Run each command on one line.

```
npm i zod react-hook-form @hookform/resolvers @better-fetch/fetch hono @hono/zod-validator
    firebase-admin pg dotenv @prisma/client@7.10.0 @prisma/adapter-pg@7.10.0
npm i -D prisma@7.10.0 @types/pg prettier
npm i tailwindcss @tailwindcss/postcss postcss
```

Step 3: Configure environment variables, Firestore and Git

Files created: `.env` (never committed) and `.env.example` (committed). Next.js loads `.env`; the Prisma CLI loads it through `dotenv`. Generate the token with `openssl rand -hex 24`.

Firebase values. Copy `project_id`, `client_email` and `private_key` from the downloaded service account JSON into `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL` and `FIREBASE_PRIVATE_KEY`. Keep the private key in double quotes on one line with the line breaks written as `\n`; `lib/server/firestore.ts` turns them back into real line breaks. Then delete the JSON file or keep it outside the project: it grants full admin access.

Create the database. In the Firebase console open Firestore and click Create database, accept the `(default)` database and choose a region. Security rules do not matter for this app, because the Admin SDK bypasses them and no browser talks to Firestore.

File updated: `.gitignore`. The starter already ignores `.env*`, which also hides the example file. Add:

```
!.env.example
/generated/prisma
*-firebase-adminsdk-*.json
serviceAccount*.json
```

Step 4: Define the database with Prisma

Files created: `prisma.config.ts` and `prisma/schema.prisma`. Prisma 7 reads the database URL from `prisma.config.ts`, writes the client to `generated/prisma`, and needs the `pg` driver adapter at runtime.

Model	What it represents	Why it is a separate model
`Form`	One survey (title, description)	Parent record; owns questions and submissions
`Question`	One prompt in a form	Needs its own type, options, order and required flag, and answers point to it
`Submission`	One respondent's attempt	Groups answers so each response can be counted, paged and deleted as a unit
`Answer`	One value for one question	Joins a submission to a question

```
npx prisma migrate dev --name init
npx prisma generate
```

Application code imports the client as `@/generated/prisma/client`. Do not drop or reset the database schema on a machine that holds real forms; a reset deletes them, and the Firestore mirror is only repopulated from Postgres, not the other way round.

Step 5: Server foundations

- `lib/server/env.ts`: validates the environment with Zod. Two lazy getters keep the database variables apart from the Firebase ones, so forms keep working when Firebase is not configured.
- `lib/server/db.ts`: one shared Prisma client with the Postgres adapter, kept on `globalThis` so hot reloading does not open new pools.
- `lib/server/firestore.ts`: starts the Firebase Admin app once (reused if it exists) and returns the Firestore database.
- `lib/server/errors.ts`: one error body shape, `{ error: { code, message } }`.
- `lib/server/methods.ts`: `methodNotAllowed`, the 405 and `Allow` helper used by every route file.
- `lib/server/validate.ts`: a wrapper around `zValidator` that returns the same 400 body with the list of issues.

Step 6: Write the schemas, one file per concern

- `lib/schemas/question.ts`: question types, the question schema and `isChoiceType`.
- `lib/schemas/form.ts`: create-form payload, form response, dashboard summary and the id parameter.
- `lib/schemas/upload.ts`: allowed file types, the 700 KB limit, the pattern a file id must match, and the upload result.
- `lib/schemas/submission.ts`: the answers payload, the page of submissions and `buildAnswerSchema`.
- `lib/schemas/builder.ts`: client-only, the one file in `lib/schemas/` the server never imports.

`z.email()` and `z.iso.date()` are Zod 4 forms; with an older Zod use `z.string().email()` and `z.string().date()`.

Step 7: Build the repository layer

- `lib/server/repo/postgres/forms.ts` and `submissions.ts`: all Prisma calls. Forms are parsed with `formSchema` so bad rows are caught on the server. `createSubmission` writes the submission and its answers in one nested create (a single transaction), skips empty answers, stores numbers as numbers, and returns what it stored so the same row can be mirrored.
- `lib/server/repo/firestore/forms.ts` and `submissions.ts`: mirror writes, mirror reads and the counter transaction described in Section 3.
- `lib/server/repo/dual.ts`: `mirrorSafely` (log, do not fail) and `readWithFallback`.
- `lib/server/repo/forms.ts` and `submissions.ts`: the facade the routes import. Postgres first, then mirror; Postgres read first, then fallback.
- `lib/server/repo/files.ts`: save, load and existence-check for uploaded files.
- `lib/server/repo/sync.ts`: `getSyncStatus` and `syncAll`.

Step 8: Build the Hono API

Method	Path	Input	Result
GET	`/api/forms`	none	200, forms with counts
POST	`/api/forms`	create-form payload	201, the form; 400
GET	`/api/forms/:id`	none	200; 404
DELETE	`/api/forms/:id`	none	204; 404
POST	`/api/forms/:id/submissions`	answers, checked against that form's questions	201, `{ id }`; 400; 404
GET	`/api/forms/:id/submissions`	`?page&limit`	200, one page; file answers are download paths
POST	`/api/files`	multipart field `file`	201, `{ id }`; 400
GET	`/api/files/:id`	none	200, the file as a download; 404
GET	`/api/sync/status`	bearer token	200, comparison of both databases; 401
POST	`/api/sync`	bearer token	200, what was copied; 401

405 for every other method. Build this once, then reuse it on every path.

1. Add a small helper in `lib/server/methods.ts`, `methodNotAllowed(app, path, allowedMethods)`. It builds the `Allow` value from the allowed methods plus `OPTIONS`, registers an `OPTIONS` handler that answers 204 with that header, and registers `app.all(path, ...)` that answers 405 with the `Allow` header and the usual error body, code `method_not_allowed`.
2. Call it at the end of each route file, after the real handlers, because Hono runs handlers in the order they were registered. Pass the methods that path really supports, for example `["GET", "POST"]` for `/api/forms`. `HEAD` is answered through `GET` automatically.

3. In `lib/server/app.ts` add `app.notFound(...)` for a JSON 404 and `app.onError(...)` for a generic 500 that logs the real error and never sends it to the client.
4. Check it: `curl -i -X DELETE /api/forms` should return 405 with `Allow: GET, POST, OPTIONS`, and an unknown path should return the JSON 404.

Error bodies look like `{ error: { code, message, issues? } }`.

Submissions. The route builds the answer schema from the form's own questions on every request, validates the body with it, checks that every file answer is the id of a file this app stored, and only then saves. A body that is not JSON becomes `null` and fails validation with a 400.

Files. The upload route re-checks type and size on the server, because the browser check can be skipped. Downloads are sent as attachments with `X-Content-Type-Options: nosniff`, so an uploaded file is never run as a page on this site.

Sync routes. They can rewrite a whole database and reveal counts, so they answer 401 unless the request carries `Authorization: Bearer <SYNC_TOKEN>`. With no token configured they stay closed.

Step 9: Mount Hono inside Next.js

`app/api/[[...route]]/route.ts` exports GET, POST, PUT, PATCH, DELETE and OPTIONS, all handled by the Hono app. All methods are exported on purpose: Next.js answers an unexported method with its own empty 405, while Hono answers with the JSON 405 and the `Allow` header. There is no `runtime` export, because Cache Components rejects it.

Step 10: Create the typed HTTP client and class patterns

`lib/fetch.ts` lists every JSON endpoint with its schemas, so a mistyped URL or wrong body is a type error and every response is validated. `getErrorMessage` reads the server's error message. File upload uses a plain `fetch` with `FormData` because a multipart body cannot be described by a Zod input schema; its response is still parsed with a schema. `lib/useLoad.ts` runs an async loader and reports loading, error or data. `lib/responsive/pattrens/forms.ts` holds every Tailwind class string, one object per component; components import their object as `p` and write `className={p.input}`. The folder keeps the spelling `pattrens` used in your other projects.

Step 11: Build the frontend

File	Responsibility
`app/page.tsx`	Dashboard: "New form" button and the list of forms.
`app/forms/new/page.tsx`	Builder page; the builder sits inside `Suspense`.
`app/f/[formId]/page.tsx`	Public survey page; params are read inside `Suspense`.
`app/forms/[id]/responses/page.tsx`	Responses page; params are read inside `Suspense`.
`app/layout.tsx`	Root layout. `<body>` has `suppressHydrationWarning` because browser extensions such as Grammarly add attributes before React loads.
`components/FormBuilder.tsx`	Create a form: react-hook-form with a field array and `builderFormSchema`.
`components/OptionsEditor.tsx`	Choices of one question: one input per option, add and remove, and a Yes / No preset.
`components/QuestionField.tsx`	One input per question type.
`components/FileUploadField.tsx`	Checks the file, uploads it to `/api/files`, stores the returned id.
`components/SurveyForm.tsx`	Fills in a survey with a schema built from its questions, then submits.
`components/SurveyLoader.tsx`	Loads a form for the public page.
`components/FormList.tsx`	Dashboard list with links to the survey and its responses.
`components/ResponsesView.tsx`	Table with one column per question and paging; file answers are links.

Step 12: Verify

```
npx prettier --check .
npx tsc --noEmit
npx eslint .
npm run build
npm run dev
```

Add a `.prettierignore` listing `.next`, `.agents`, `.claude`, `generated`, `node_modules` and `package-lock.json` so Prettier skips generated files. Do not run `npm run build` while `npm run dev` is serving the same folder: the build replaces the `.next` directory and can stop the dev server.

Then test by hand: create a form with one question of each type, open its public link, submit an invalid answer (an error appears next to that question), submit a valid one, and open the responses page.

Step 13: Operate the two databases

```
curl -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync/status
curl -X POST -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync
```

`status` lists form ids missing from Firestore, ids that exist only in Firestore, and both submission counts; `inSync` is true only when all of these match. Comparing ids matters: counts alone say "in sync" when one form was deleted and another added. `POST /api/sync` copies every form and

submission from Postgres (documents are replaced, so repeating it is safe), sets each form's counter to the real number, and removes Firestore forms that no longer exist in Postgres. Run it after any Firestore outage, after restoring Postgres from a backup, or when the server log shows "Firestore mirror failed".

5 What Was Verified

Checked in this project:

- `prettier -check`, `tsc -noEmit`, `eslint .` and `npm run build` all pass.
- API, against a running dev server with a real Postgres (Docker) and a real Firestore database: creating forms; rejecting a choice question with one option; rejecting invalid answers with one message per field; saving a valid submission and listing it with numbers stored as numbers; dashboard counts; 404 and 405 bodies.
- Files: upload of a text file, download as an attachment with `nosniff`, rejection of a disallowed type and of an 800 KB file, rejection of a malformed and of an unknown file id on submit, acceptance of a real id.
- Sync: new writes appear in both databases with equal counts; a form deleted from Firestore and a stray form added were both reported and repaired; deleting a form removes it from both; with the Postgres container stopped, the form list, a form and its submissions were still served from Firestore, and the databases matched again after Postgres returned.
- Browser: the builder creates a multi-choice form using the Yes / No preset; the public survey page renders the question and options; the hydration warning caused by a browser extension no longer shows.

Not checked:

- A full click-through of every question type in the browser (date and number inputs, the file input and the validation messages shown next to fields). They were checked through the API and the shared schemas only.
- The "Firestore mirror failed" path with a real Firestore error. Drift was created by editing Firestore by hand, not by a failing write.
- Deployment, concurrent writers, large data volumes, and a Firebase Storage bucket (not used).

6 Known Limits

- **No login.** Anyone with a form link can answer it, and the forms list, responses and file downloads need no sign-in. File ids are random and unguessable, but the responses endpoint returns them.
- **File size.** Files are limited to 700 KB because they are stored inside Firestore documents. Photos and short PDFs fit; larger files need a real bucket.
- **Orphaned files.** Deleting a form does not delete the files its answers pointed to.
- **One-way sync and manual repair**, as described in Section 3.
- **No Delete button.** Forms can only be deleted through the API.

7 Source Code

Every listing below is read directly from the repository when this document is built, so it is the exact content of the file. The code follows these conventions: braces on every `if`, a doc comment on each function, descriptive names, Prettier width 80 with single quotes and trailing commas, and no inline Tailwind classes (they live in the patterns file). Comments marked "Why" explain a decision, not the syntax.

7.1 Configuration

File: `prisma.config.ts`. Prisma 7 configuration: schema path, migrations path and database URL.

(file not created yet)

File: `prisma/schema.prisma`. The four models, with a comment on why each exists.

```
// This is your Prisma schema file,
// learn more about it in the docs: https://pris.ly/d/prisma-schema

// Get a free hosted Postgres database in seconds: `npx create-db`

generator client {
  provider = "prisma-client"
  output   = "../app/generated/prisma"
}

datasource db {
  provider = "postgresql"
}
```

File: `postcss.config.mjs`. Tailwind v4 PostCSS plugin.

(file not created yet)

File: `.env.example`. Environment variable template.

```
// Database connection string for PostgreSQL
DATABASE_URL=postgresql://user:pass@host:5432/surveys
// Firebase service account credentials
FIREBASE_PROJECT_ID=
// The client email is the service account email address, e.g. "
FIREBASE_CLIENT_EMAIL=
# The private key must be a single line string, with newlines replaced by \n. For example:
FIREBASE_PRIVATE_KEY=
# bearer token for POST /api/sync and GET /api/sync/status
SYNC_TOKEN=
```

File: `.prettierrc`. Prettier settings: width 80, single quotes, trailing commas.

(file not created yet)

7.2 Shared API schemas

File: `lib/schemas/question.ts`. Question types, schema and helper.

(file not created yet)

File: `lib/schemas/form.ts`. Create-form payload, form response, dashboard summary.

(file not created yet)

File: `lib/schemas/upload.ts`. Upload limits, allowed types, file id pattern.

(file not created yet)

File: `lib/schemas/submission.ts`. Answers, submissions page, and the runtime answer schema.

(file not created yet)

7.3 Server foundations

File: `lib/server/env.ts`. Lazy, validated environment variables.

(file not created yet)

File: `lib/server/db.ts`. Shared Prisma client with the Postgres adapter.

(file not created yet)

File: `lib/server/firestore.ts`. Firebase Admin app and Firestore database.

(file not created yet)

File: `lib/server/errors.ts`. The one error body shape.

(file not created yet)

File: `lib/server/methods.ts`. Helper that adds the 405 + Allow catch-all to a path.

(file not created yet)

File: `lib/server/validate.ts`. zValidator wrapper with a consistent 400 body.

(file not created yet)

7.4 Repository layer

File: `lib/server/repo/postgres/forms.ts`. Form data access in Postgres (source of truth).

(file not created yet)

File: `lib/server/repo/postgres/submissions.ts`. Submission data access in Postgres.

(file not created yet)

File: `lib/server/repo/firestore/forms.ts`. Form mirror writes and reads in Firestore.

(file not created yet)

File: `lib/server/repo/firestore/submissions.ts`. Submission mirror writes and reads in Firestore.

(file not created yet)

File: `lib/server/repo/dual.ts`. Safe mirror write and read with fallback.

(file not created yet)

File: `lib/server/repo/forms.ts`. Form facade: Postgres first, then mirror.

(file not created yet)

File: `lib/server/repo/submissions.ts`. Submission facade: Postgres first, then mirror.

(file not created yet)

File: `lib/server/repo/files.ts`. Uploaded files stored in Firestore.

(file not created yet)

File: `lib/server/repo/sync.ts`. Sync status and full repair.

(file not created yet)

7.5 API routes

File: `lib/server/routes/forms.ts`. Form endpoints and 405 handling.

(file not created yet)

File: `lib/server/routes/submissions.ts`. Submission endpoints and 405 handling.

(file not created yet)

File: `lib/server/routes/files.ts`. File upload and download.

(file not created yet)

File: `lib/server/routes/sync.ts`. Token-protected sync endpoints.

(file not created yet)

File: `lib/server/app.ts`. The Hono app: routes, 404 and error handler.

(file not created yet)

File: `app/api/[[...route]]/route.ts`. Mounts Hono inside Next.js.

(file not created yet)

7.6 Client library

File: `lib/fetch.ts`. Typed HTTP client and error message helper.

(file not created yet)

File: `lib/useLoad.ts`. Loading hook.

(file not created yet)

File: `lib/schemas/builder.ts`. Builder-only schema and conversion to the API payload.

(file not created yet)

File: `lib/responsive/pattrens/forms.ts`. All Tailwind class strings.

(file not created yet)

7.7 Components

File: `components/FormBuilder.tsx`. Create-form screen.

(file not created yet)

File: `components/OptionsEditor.tsx`. Options editor for choice questions.

(file not created yet)

File: `components/QuestionField.tsx`. One input per question type.

(file not created yet)

File: `components/FileUploadField.tsx`. File picker and upload.

(file not created yet)

File: `components/SurveyForm.tsx`. Public survey form.

(file not created yet)

File: `components/SurveyLoader.tsx`. Loads a form for the public page.

(file not created yet)

File: `components/FormList.tsx`. Dashboard list.

(file not created yet)

File: `components/ResponsesView.tsx`. Responses table with paging.

(file not created yet)

7.8 Pages

File: `app/layout.tsx`. Root layout.

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Create Next App",
  description: "Generated by create next app",
};

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col">{children}</body>
    </html>
  );
}
```

File: `app/page.tsx`. Dashboard.

```
import Image from "next/image";

export default function Home() {
  return (
    <div className="flex flex-col flex-1 items-center justify-center bg-zinc-50 font-sans dark:bg-
    black">
      <main className="flex flex-1 w-full max-w-3xl flex-col items-center justify-between py-32 px
    -16 bg-white dark:bg-black sm:items-start">
        <Image
          className="dark:invert h-5 w-[100px]"
          src="/next.svg"
```

```
      alt="Next.js logo"
      width={100}
      height={20}
      priority
    />
    <div className="flex flex-col items-center gap-6 text-center sm:items-start sm:text-left">
      <h1 className="max-w-xs text-3xl font-semibold leading-10 tracking-tight text-black dark
:text-zinc-50">
        To get started, edit the{" "}
        <code className="rounded bg-black/[.06] px-1.5 py-0.5 font-mono text-[0.9em] dark:bg-
white/[.08]">
          page.tsx
        </code>{" "}
        file.
      </h1>
      <p className="max-w-md text-lg leading-8 text-zinc-600 dark:text-zinc-400">
        Looking for a starting point or more instructions? Head over to{" "}
        <a
          href="https://vercel.com/templates?framework=next.js&utm_source=create-next-app&
utm_medium=appdir-template-tw&utm_campaign=create-next-app"
          className="font-medium text-zinc-950 dark:text-zinc-50"
        >
          Templates
        </a>{" "}
        or the{" "}
        <a
          href="https://nextjs.org/learn?utm_source=create-next-app&utm_medium=appdir-template
-tw&utm_campaign=create-next-app"
          className="font-medium text-zinc-950 dark:text-zinc-50"
        >
          Learning
        </a>{" "}
        center.
      </p>
    </div>
    <div className="flex flex-col gap-4 text-base font-medium sm:flex-row">
      <a
        className="flex h-12 w-full items-center justify-center gap-2 rounded-full bg-
foreground px-5 text-background transition-colors hover:bg-[#383838] dark:hover:bg-[#ccc] md:w
-[158px]"
        href="https://vercel.com/new?utm_source=create-next-app&utm_medium=appdir-template-tw&
utm_campaign=create-next-app"
        target="_blank"
        rel="noopener noreferrer"
      >
        <Image
          className="dark:invert h-[14px] w-4"
          src="/vercel.svg"
          alt="Vercel logomark"
          width={16}
          height={14}
        />
        Deploy Now
      </a>
      <a
        className="flex h-12 w-full items-center justify-center rounded-full border border-
solid border-black/[.08] px-5 transition-colors hover:border-transparent hover:bg-black/[.04]
dark:border-white/[.145] dark:hover:bg-[#1a1a1a] md:w-[158px]"
        href="https://nextjs.org/docs?utm_source=create-next-app&utm_medium=appdir-template-tw
&utm_campaign=create-next-app"
        target="_blank"
        rel="noopener noreferrer"
      >
        Documentation
      </a>
```

```
        </div>
      </main>
    </div>
  );
}
```

File: `app/forms/new/page.tsx`. Builder page.

(file not created yet)

File: `app/f/[formId]/page.tsx`. Public survey page.

(file not created yet)

File: `app/forms/[id]/responses/page.tsx`. Responses page.

(file not created yet)